INF266 - Problem Set 4

Ragnhild Klette, Maya Evensen, Halvard Håvardsrud

March 9, 2026

Part I - Problems

Exercise 1

1. (*) When using a linear functions for feature engineering you transform the original state into features. This creates a new feature vector from the raw input. The purpose is to represent the state in a more useful way for the model. For linear approximation on the other hand the goal is to approximate the target function.
$$f(s; w) = w^T \phi(s)$$
The parameters w are learned from data. It maps features to predictions.

2. Let the approximate value function be $\hat{v}(s; w) = w^\top \phi(s)$ The TD(n) target is
$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}(S_{t+n}; w)$$
The semi-gradient update for the weights is
$$w \leftarrow w + \alpha\big(G_t^{(n)} - \hat{v}(S_t; w)\big)\nabla_w \hat{v}(S_t; w)$$
For linear approximation $\nabla_w \hat{v}(S_t; w) = \phi(S_t)$, giving
$$w \leftarrow w + \alpha\big(G_t^{(n)} - \hat{v}(S_t; w)\big)\phi(S_t)$$

3. (*) The policy improvement theorem assumes access to the exact action-value function $q_\pi(s, a)$. With function approximation we only have an estimate $\hat{q}(s, a)$, which may contain errors. A policy that is greedy with respect to $\hat{q}$ may therefore choose actions that are not actually better. Consequently, the step in the proof assuming $q_\pi(s, \pi'(s)) \geq q_\pi(s, \pi(s))$ no longer holds, and policy improvement is not guaranteed.

4. (*) Replacing the tabular action-value function with a neural network introduces the challenge of combining function approximation, bootstrapping, and off-policy learning. This combination is known to cause instability in reinforcement learning.

   In the tabular case, each update affects only a single state–action value, which guarantees stable learning. With a neural network, however, updates affect many states simultaneously due to shared parameters. When combined with TD(λ) bootstrapping and off-policy data, errors can propagate through the value estimates and may lead to unstable learning or even divergence.

Exercise 2

1. MC values: From uni, the episode ends immediately after the exam, so return equals exam reward:
$$v_{\pi_{MC}}(\text{uni}) = 86 \cdot \frac{1 + 2 \cdot 0}{86} = 0.75$$
Home appears only once, so return from home is:
$$G(\text{home}) = 0 + \gamma \cdot 0 = 0$$
Results MC:
$$v_{\pi_{MC}}(\text{uni}) = 0.75, \quad v_{\pi_{MC}}(\text{home}) = 0$$

2. TD values: From uni, taking exam terminates, so the TD target is just the immediate reward:

$$v_{\pi_{TD}}(\text{uni}) \approx E[R \mid \text{uni}, \text{take exam}] = 0.75$$

The only observed transition from home is:

$$v(\text{home}) \approx 0 + \gamma v(\text{uni}) = \gamma \cdot 0.75$$

Results TD:

$$v_{\pi_{TD}}(\text{uni}) = 0.75, \quad v_{\pi_{TD}}(\text{home}) = 0.75\gamma$$

3. (*) As calculated above, the state values are not identical unless $\gamma = 0$. This is because MC gives $v(home) = 0$ because it uses the single observed return from home (and that episode got exam reward 0). TD gives $v(home) = 0.75\gamma$ because it bootstraps from $v(uni)$, which is estimated from many uni episodes. TD therefore propagates the typical uni outcome back to home.

4. A full learned model is: $\hat{M} = \langle S, A, \hat{T}, \hat{R}, \gamma \rangle$. Since S, A, and γ are already given, we need to learn the transition function $\hat{T}$ and the reward function $\hat{R}$.

5. The transition function is defined as: $T(s, a, s') = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$

6. From the observed trajectories, we obtain:

$$T(\text{home}, \text{travel}, \text{uni}) = 1$$

$$T(\text{uni}, \text{take exam}, \text{terminal}) = 1$$

So the transition table is:

s	a	s'
home	travel	uni
uni	take exam	terminal

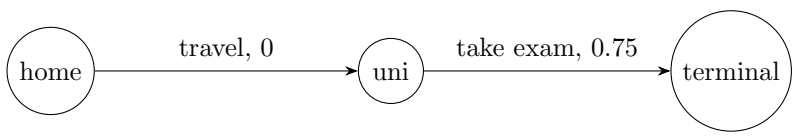
7. The reward function is defined as: $R(s, a) = E[R_{t+1} \mid S_t = s, A_t = a]$

8. For (uni, take exam), the observed rewards are $1, 1, 0, 1, 1, 1, 1, 0$. Hence $R(\text{uni}, \text{take exam}) = \frac{6}{8} = 0.75$. For (home, travel), the observed reward is 0, hence $R(\text{home}, \text{travel}) = 0$. So the reward table is:

s	a	$R(s, a)$
home	travel	0
uni	take exam	0.75

9. The reward function $R(s, a)$ gives the immediate expected reward after taking action a in state s. The value function $v_\pi(s)$ gives the expected discounted sum if future rewards when starting in state s and following policy π. Therefore, v_π includes long-term consequences, while R only describes the next reward.

10. The learned MDP graph is shown below:



11. Using the Bellman equations for the learned model:

$$v_\pi(\text{uni}) = R(\text{uni}, \text{take exam}) = 0.75$$

$$v_\pi(\text{home}) = R(\text{home}, \text{travel}) + \gamma v_\pi(\text{uni}) = 0 + \gamma \cdot 0.75 = 0.75\gamma$$

Therefore:

$$v_\pi^{DP}(\text{uni}) = 0.75, \quad v_\pi^{DP}(\text{home}) = 0.75\gamma$$

12. (*) The DP values are $v_\pi^{DP}(\text{uni}) = 0.75$ and $v_\pi^{DP}(\text{home}) = 0.75\gamma$.

    These are identical to the TD values, but differ from the MC estimate for home, which was 0. The reason is that MC uses the single observed return from the only trajectory starting at home, while TD and DP use the transition from home to uni together with the value of uni. Therefore, TD and DP propagate the value of uni back home, whereas MC depends directly on the sampled return in the observed trajectory.

13. Using both the real and simulated trajectories, the state uni is visited 16 times in total. The observed exam rewards are 12 ones and 4 zeros, so: $v_\pi^{MC}(\text{uni}) = \frac{12}{16} = 0.75$. The state home is visited 4 times in total: once in the real data and three times in the simulated data. The corresponding returns are $0, \gamma, \gamma, 0$.

    Therefore:

$$v_\pi^{MC}(\text{home}) = \frac{0 + \gamma + \gamma + 0}{4} = \frac{2\gamma}{4} = 0.5\gamma$$

14. For uni, taking the action 'take exam' terminated the episode, so the TD estimate is the expected immediate reward: $v_\pi^{TD}(\text{uni}) = 0.75$

    For home, the observed transition is still: home $\rightarrow$ uni with reward 0. Therefore, TD bootstraps from the value of uni:

$$v_\pi^{TD}(\text{home}) = 0 + \gamma v_\pi^{TD}(\text{uni}) = 0.75\gamma$$

    This gives us $v_\pi^{TD}(\text{uni}) = 0.75$ and $v_\pi^{TD}(\text{home}) = 0.75\gamma$.

15. (*) The value of uni did not change for either MC or TD. In both the real and simulated data, the empirical reward distribution at uni remained the same: 75% of the exams produced reward 1 and 25% produced reward 0. Therefore: $v_\pi(uni) = 0.75$ for both methods.

    The value of home changes for MC but not for TD. Previously, MC estimated $v_\pi(home) = 0$ because only one trajectory starting from home was observed, and that trajectory produces total return 0. After adding simulated data, more trajectories starting from home were available, including some with return γ. As a result, the MC estimate increased to:

$$v_\pi^{MC}(home) = 0.5\gamma$$

    The TD estimate remained:

$$v_\pi^{TD}(home) = 0.75\gamma$$

    because TD bootstraps from the value of uni, which did not change. Therefore, simulations changed the MC estimate by providing more complete sampled returns from home, while the TD estimate stayed the same since it already relied on the transition to uni and the estiated value of uni.

Exercise 3

1. (*) Monte Carlo Tree Search (MCTS) can improve decision making by simulating many possible future trajectories and selecting actions based on their expected outcomes. However, several considerations are necessary before applying it to a self-driving car problem.

   First, MCTS requires a model or simulator of the environment in order to generate simulated trajectories. The quality of the resulting policy therefore depends on the accuracy of this model. Second, MCTS can be computationally expensive because it requires many simulations for each decision. In a self-driving scenario, decisions need to be taken quickly in real time, which may limit the number of simulations that can be performed.

   Finally, driving environments typically have continuous state and action spaces, while MCTS is more naturally suited to discrete decision problems. For these reasons, the use of MCTS may be beneficial only if the environment can be simulated efficiently and the decision space can be suitably discretized.

Part II - Report

Exercise 4

1. (Done in code). In MountainCar, you get -1 reward per step until you reach the goal.

2. (*) The results show that the discretization step size Δ strongly affects learning performance. Very large Δ (few bins) leads to poor policies due to loss of state information, while very small Δ (many bins) increases the size of the state space and slows learning, each state is visited less frequently during training.

   In this task, a discretization of (40,40) bins provides the best trade-off between representation accuracy and learning efficiency, achieving the highest average return.

n_{bins}	Δ_{pos}	Δ_{vel}	Eval Return ($\pm$ std)
(10,10)	0.1800	0.014000	-200.00 ± 0.00
(20,20)	0.0900	0.007000	-182.70 ± 25.25
(30,30)	0.0600	0.004667	-168.28 ± 19.37
(40,40)	0.0450	0.003500	-149.66 ± 30.05
(50,50)	0.0360	0.002800	-187.94 ± 18.79

Table 1: Performance of SARSA for different discretization resolutions in the MountainCar environment.

3. (*) Yes, we could try normalizing and linear function approximation. Just scale position and velocity to [-1,1] (or mean 0, std 1), then learn with a linear approximator. It might be simple, but it helps stability and makes step sizes easier to tune.

   Another possible improvement is to use multiple offset grids (tilings) instead of a single discretization grid. The state is then represented by the set of active tiles across these tilings. This approach provides smoother generalization than simple binning while still allowing the use of linear or tabular-style updates, and it is commonly applied in problems such as MountainCar.

Exercise 5

1. (Done in code)

2. (Done in code)

3. After training the Dyna-Q agent, we derive a policy by selecting the action with the highest Q-value in each state (a greedy policy). We then simulate a trajectory starting from the initial state by following this policy until the landing zone is reached. This trajectory represents the optimal path according to the learned Q-function.

Figure 1: Optimal trajectory learned by Dyna

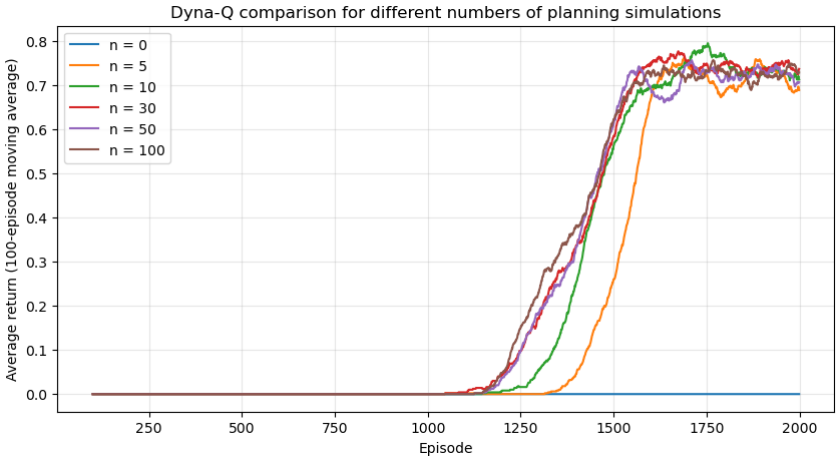


Figure 2: comparison for different numbers of planning simulations

planning n	**mean last100**	**std last100**
0	0.000	0.000
5	0.688	0.040
10	0.712	0.013
30	0.736	0.046
50	0.706	0.045
100	0.728	0.024

Table 2: Performance statistics for different planning steps

4. (*) We evaluated Dyna-Q with different numbers of planning simulations per real interaction step, using $n \in 1, 5, 10, 30, 50, 100$. For each setting we ran multiple seeds and compared the average return over training episodes. The results show that increasing the number of planning steps significantly improves sample efficiency. When n=0, the algorithm reduces to standard Q-learning and learns much more slowly. As n increases, the agent is able to exploit the learned model to perform additional updates from simulated experience, leading to faster discovery of successful trajectories and earlier convergence. Beyond a certain point, increasing n further provides smaller gains, suggesting diminishing returns from additional planning.